

Ping and Watchdog

Yuval baror 214329633
Joshua Gordon 332307073

How to run the program:

1. Download the following files:
 - a. Ping.py
 - b. Better_ping.py
 - c. Watchdog.py
2. Put all the files in a single directory.
3. Open the directory in the linux terminal
4. Run the following commands:
 - a. For part 1: `sudo python3 ping.py <ip>`
 - b. For part 2 : `sudo python3 better_ping.py <ip>`
5. In order to close the parts press Ctrl+c

System overview:

Part 1:

We'll implement the "ping" command by creating a program called "ping.py" that takes an argument specifying the host to ping. The purpose of the ping command is to check the connection between two machines.

In the Internet protocol suite, the Internet Control Message Protocol (ICMP) serves as a supporting protocol. It enables network devices, including routers, to send error messages and operational information indicating the success or failure of communication with another IP address. For example, an error is indicated when a requested service is unavailable or when a host or router cannot be reached. Unlike transport protocols such as TCP and UDP, ICMP is not typically used to exchange data between systems and is not regularly employed by end-user network applications, except for certain diagnostic tools like ping and traceroute.

The ICMP echo request and ICMP echo reply to messages, commonly known as ping messages, are utilized. Ping is a troubleshooting tool used by system administrators to manually test connectivity between network devices and to measure network delay and packet loss.

In our program, we will send an ICMP ECHO REQUEST to the specified host. Upon receiving an ICMP ECHO REPLY, the program will continue sending subsequent ICMP ECHO REQUEST messages without stopping.

Part 2:

A watchdog is a timer designed to detect and recover from computer malfunctions or hardware failures. It consists of a chip that receives a signal every millisecond from the CPU. If the watchdog chip doesn't receive any signal for 10 seconds, typically indicating a hardware failure, it will initiate a system reboot.

To enhance the ping program, we will create a watchdog component. This watchdog will maintain a timer by establishing a TCP connection on port 3000. Its purpose is to ensure that if we don't receive an ICMP ECHO REPLY within 10 seconds after sending an ICMP REQUEST, the watchdog will terminate the program.

To incorporate this functionality, we will modify the ping.py program. We will execute the watchdog.py program alongside it. Whenever the better_ping.py program sends a packet, we will update the timer in the watchdog.py program accordingly.

Our code and outputs:

Part 1:

Ping.py –

Imports:

```
import socket
import sys
import time
import io
import struct
```

Calculate the checksum:

The purpose of the `calculate\_checksum` function is to determine the checksum value for a provided data packet. This checksum value is utilized in network protocols to verify the integrity of data while it is being transmitted. To calculate the checksum, the function divides the data into 16-bit words, adds them together, performs carry propagation if necessary, and then calculates the one's complement to obtain the final checksum value. By performing these steps, the function ensures that the data's integrity can be validated during transmission.

```
def calculate_checksum(data):
    checksum = 0
    if len(data) % 2 != 0: # if odd number of bytes
        data += b'\x00'
    for i in range(0, len(data), 2): # iterate over bytes pairwise
        chunk = data[i:i+2] # get 2 bytes
        word = struct.unpack('!H', chunk)[0]
        checksum += word
    checksum = (checksum >> 16) + (checksum & 0xFFFF)
    checksum += checksum >> 16
    checksum = ~checksum & 0xFFFF
    return checksum
```

Create ping package:

The purpose of the `create\_ping\_packet` function is to generate an ICMP ping packet, which is used for checking connectivity and measuring round-trip time between network devices using the ICMP (Internet Control Message Protocol) protocol. In short, the function sets the relevant fields in the ICMP header, calculates the checksum, and constructs the ICMP packet by combining the header fields. The resulting ICMP packet is then returned for utilization in sending ICMP ping requests.

```
def create_ping_packet():
    icmp_type = 8
    icmp_code = 0
    icmp_checksum = 0
    icmp_identifier = 1
    icmp_sequence = 1
    icmp_header = struct.pack('!BBHHH', icmp_type, icmp_code, icmp_checksum, icmp_identifier, icmp_sequence)
    icmp_checksum = calculate_checksum(icmp_header)
    icmp_header = struct.pack('!BBHHH', icmp_type, icmp_code, icmp_checksum, icmp_identifier, icmp_sequence)
    icmp_packet = icmp_header
    return icmp_packet
```

The main function – handle ping:

The purpose of the `handle\_ping` function is to send ICMP ping requests and receive ICMP ping reply packets. It makes use of the `create\_ping\_packet` function to create the ICMP ping packet. In short, the function operates in a loop, continuously sending ICMP ping requests and receiving the corresponding ICMP ping reply packets. It then prints the relevant information, such as the IP address, sequence number, and round-trip time. Additionally, it handles a keyboard interruption by ensuring that the socket is closed if it is open.

```
def handle_ping():
    try:
        Packet = create_ping_packet()
        with socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP) as sock:
            seq = 0
            while True:
                start = time.time()
                sock.sendto(Packet, (sys.argv[1], 8))
                pong_pkt = sock.recvfrom(io.DEFAULT_BUFFER_SIZE)
                end = time.time() # time in seconds
                time.sleep(0.5)
                print(f"Packet IP: {pong_pkt[1][0]} , seq : {seq} , time : {end-start} seconds")
                seq = seq + 1
    except KeyboardInterrupt:
        if sock: # if socket is open close it
            sock.close()
```

Putput:

The program will continue sending ping messages until the user terminates it by pressing Ctrl+C.

```
Packet IP: 8.8.8.8 , seq : 10 , time : 0.016707658767700195 seconds
Packet IP: 8.8.8.8 , seq : 11 , time : 0.015839338302612305 seconds
Packet IP: 8.8.8.8 , seq : 12 , time : 0.015232563018798828 seconds
Packet IP: 8.8.8.8 , seq : 13 , time : 0.015721797943115234 seconds
Packet IP: 8.8.8.8 , seq : 14 , time : 0.016788005828857422 seconds
Packet IP: 8.8.8.8 , seq : 15 , time : 0.016350507736206055 seconds
Packet IP: 8.8.8.8 , seq : 16 , time : 0.018087148666381836 seconds
Packet IP: 8.8.8.8 , seq : 17 , time : 0.016930580139160156 seconds
Packet IP: 8.8.8.8 , seq : 18 , time : 0.014750480651855469 seconds
Packet IP: 8.8.8.8 , seq : 19 , time : 0.01554417610168457 seconds
Packet IP: 8.8.8.8 , seq : 20 , time : 0.013885736465454102 seconds
Packet IP: 8.8.8.8 , seq : 21 , time : 0.013755321502685547 seconds
Packet IP: 8.8.8.8 , seq : 22 , time : 0.016060590744018555 seconds
Packet IP: 8.8.8.8 , seq : 23 , time : 0.013890266418457031 seconds
Packet IP: 8.8.8.8 , seq : 24 , time : 0.011112689971923828 seconds
Packet IP: 8.8.8.8 , seq : 25 , time : 0.01696491241455078 seconds
Packet IP: 8.8.8.8 , seq : 26 , time : 0.017779111862182617 seconds
Packet IP: 8.8.8.8 , seq : 27 , time : 0.010975122451782227 seconds
Packet IP: 8.8.8.8 , seq : 28 , time : 0.016978025436401367 seconds
Packet IP: 8.8.8.8 , seq : 29 , time : 0.011731863021850586 seconds
Packet IP: 8.8.8.8 , seq : 30 , time : 0.015329837799072266 seconds
Packet IP: 8.8.8.8 , seq : 31 , time : 0.013315200805664062 seconds
Packet IP: 8.8.8.8 , seq : 32 , time : 0.014895200729370117 seconds
^Cyual@yuval:~/Desktop/computer_networks/ex4/PING---Sockets_TCP_UDP-main$
```

Part 2:

Better ping.py –

Imports:

```
import signal
import sys
from socket import *
import os
import time
import struct
```

Calculate the checksum:

The purpose of the `checksum` function is to compute the checksum value for a provided data packet. The checksum value is utilized in network protocols to verify the integrity of data during its transmission.

```
def checksum(pkt) -> int:
    """
    Calculate the checksum of the given data.

    """
    res = 0

    if len(pkt) % 2 != 0:
        pkt += b'\x00'
    for i in range(0, len(pkt), 2):
        chunk = pkt[i:i + 2]
        word = struct.unpack('!H', chunk)[0]
        res += word
    res = (res >> 16) + (res & 0xFFFF)
    res += res >> 16
    # One's complement
    res = ~res & 0xFFFF
    return res
```

Create ping packets:

Same function as in “ping.py”

```
def ping_packet() -> bytes:
    """
    Create an ICMP ping packet.

    """
    icmp_type = 8 # ICMP Echo Request
    icmp_code = 0
    icmp_checksum = 0
    icmp_identifier = 18 # TODO:
    icmp_sequence = 1
    icmp_header = struct.pack('!BBHHH', icmp_type, icmp_code, icmp_checksum, icmp_identifier, icmp_sequence)
    icmp_checksum = checksum(icmp_header)
    icmp_header = struct.pack('!BBHHH', icmp_type, icmp_code, icmp_checksum, icmp_identifier, icmp_sequence)
    icmp_packet = icmp_header

    return icmp_packet
```

Main function- handle better ping:

The `handle\_better\_ping` function is responsible for managing the improved ping functionality. In summary, it performs the following steps:

1. Send ICMP ping packets.
2. Start a watchdog process.
3. Establish a TCP connection with the watchdog.
4. Receive ICMP pong packets.
5. Print the packet information.
6. Shutdown the TCP connection.
7. Terminate the watchdog process.

By carrying out these steps, the `handle\_better\_ping` function ensures a more reliable and controlled ping operation.

```
def handle_better_ping() -> None:
    """
    Handle the betterPing logic.

    This function sends ICMP ping packets, starts a watchdog process,
    establishes a TCP connection to the watchdog, receives ICMP pong packets,
    prints the packet information, shuts down the TCP connection, and terminates
    the watchdog process.

    """
    num = 0
    with socket(AF_INET, SOCK_RAW, IPPROTO_ICMP) as sock:
        while True:
            num += 1
            ping = ping_packet()
            start = time.time()
            sock.sendto(ping, (sys.argv[1], 0))
            pid = os.fork() # Create a child process
            if pid == 0: # child process
                os.execvp("python3", ["python3", "watchdog.py"]) # Start watchdog process , execvp replace
                # current process with the watchdog process
            else: # parent process pid == child process id
                time.sleep(1) # Wait for watchdog to start
                with socket(AF_INET, SOCK_STREAM, IPPROTO_TCP) as watchdog: # watchdog socket
                    watchdog.connect(("127.0.0.1", 3000)) # Connect to watchdog
                    watchdog.sendall(sys.argv[1].encode()) # Send IP address to watchdog
                    pong = sock.recvfrom(8192) # Receive ICMP pong packet
                    end = time.time() # Get end time
                    print(
                        f"Reply from ip: {pong[1][0]}, seq num: {num}, time: {(end - start) - 1} seconds")
                    watchdog.shutdown(SHUT_RDWR) # Shutdown watchdog
                    os.kill(pid, signal.SIGTERM) # Terminate watchdog
                    os.wait() # Wait for watchdog to terminate
```

Watchdog.py –

Imports:

```
import os
import signal
import time
from socket import *
```

Main function- run watchdog:

The `run\_watchdog` function is the main function responsible for running the watchdog server. In summary, it performs the following steps:

1. Create a TCP socket using `socket(AF\_INET, SOCK\_STREAM, IPPROTO\_TCP)`.
2. Set the socket option `SO\_REUSEADDR` to 1, allowing the reuse of the address.
3. Bind the socket to the address "127.0.0.1" and port 3000 using `sock.bind()`.
4. Start listening for incoming connections using `sock.listen()`.
5. Accept a client connection, which blocks until a client connects. Obtain the client's socket and address.
6. Receive the IP address sent by the client through the client socket.
7. Wait for 10 seconds using a loop and `time.sleep(1)` to pause the execution.
8. Shutdown the client socket to close the connection.
9. Print a message indicating that the server with the received IP address cannot be reached.
10. Terminate the parent process by sending the `SIGTERM` signal to its process ID.

In summary, the `run\_watchdog` function sets up the watchdog server, receives an IP address from the client, waits for 10 seconds, and handles the termination of the parent process if the server cannot be reached.

```
def run_watchdog() -> None:
    """
    Main function to run the server.

    This function sets up a TCP socket, listens for incoming connections,
    receives an IP address from the client, waits for 10 seconds, and then
    shuts down the client connection. If the server cannot be reached, it
    terminates the parent process.

    """
    try:
        with socket(AF_INET, SOCK_STREAM, IPPROTO_TCP) as sock:
            sock.setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)
            sock.bind(("127.0.0.1", 3000))
            sock.listen()
            client, addr = sock.accept()
            ip = client.recv(8192)
            for i in range(10): # Wait for 10 seconds
                time.sleep(1)
            client.shutdown(SHUT_RDWR)
            print(f"Server {ip.decode()} cannot be reached")
            os.kill(os.getppid(), signal.SIGTERM) # Terminate the parent process
    except KeyboardInterrupt:
        pass
```

Output:

Measuring the duration between each ICMP request and response, and notifying when the server is unreachable (taking more than 10 seconds to respond).

```
yuval@yuval: ~/Desktop/computer networks/ex4/PING---Soc...  
Reply from ip: 8.8.8.8, seq num: 23, time: 0.0064046382904052734 seconds  
Reply from ip: 8.8.8.8, seq num: 24, time: 0.0051326751708984375 seconds  
Reply from ip: 8.8.8.8, seq num: 25, time: 0.004871368408203125 seconds  
Reply from ip: 8.8.8.8, seq num: 26, time: 0.005540370941162109 seconds  
Reply from ip: 8.8.8.8, seq num: 27, time: 0.003011941909790039 seconds  
Reply from ip: 8.8.8.8, seq num: 28, time: 0.003239870071411133 seconds  
Reply from ip: 8.8.8.8, seq num: 29, time: 0.0057141780853271484 seconds  
Reply from ip: 8.8.8.8, seq num: 30, time: 0.004567861557006836 seconds  
Reply from ip: 8.8.8.8, seq num: 31, time: 0.004232645034790039 seconds  
Reply from ip: 8.8.8.8, seq num: 32, time: 0.005769968032836914 seconds  
Reply from ip: 8.8.8.8, seq num: 33, time: 0.00327301025390625 seconds  
Reply from ip: 8.8.8.8, seq num: 34, time: 0.0035228729248046875 seconds  
Reply from ip: 8.8.8.8, seq num: 35, time: 0.0034813880920410156 seconds  
Reply from ip: 8.8.8.8, seq num: 36, time: 0.0028471946716308594 seconds  
Reply from ip: 8.8.8.8, seq num: 37, time: 0.0029418468475341797 seconds  
Reply from ip: 8.8.8.8, seq num: 38, time: 0.003270864486694336 seconds  
Reply from ip: 8.8.8.8, seq num: 39, time: 0.007517814636230469 seconds  
Reply from ip: 8.8.8.8, seq num: 40, time: 0.003329038619995117 seconds  
Reply from ip: 8.8.8.8, seq num: 41, time: 0.005641937255859375 seconds  
Reply from ip: 8.8.8.8, seq num: 42, time: 0.006009578704833984 seconds  
Reply from ip: 8.8.8.8, seq num: 43, time: 0.004324197769165039 seconds  
Server 8.8.8.8 cannot be reached  
Terminated
```

Wireshark recordings:

All our recordings were filtered using “icmp” on the filter line.

Part 1:

Using a correct IP address (8.8.8.8)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 2)
2	0.012968798	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 1)
3	0.515218758	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 4)
4	0.531556212	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 3)
5	1.034151913	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 6)
6	1.049917992	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 5)
7	1.551296425	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 8)
8	1.563836440	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 7)
9	2.068955404	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 10)
10	2.080731162	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 9)
11	2.585283550	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 12)
12	2.598122915	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 11)
13	3.101884304	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 14)
14	3.113982033	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 13)
15	3.614959865	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 16)
16	3.629076455	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 15)
17	4.130380893	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 18)
18	4.144831873	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 17)
19	4.650757397	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 20)
20	4.664075063	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 19)
21	5.165019363	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 22)
22	5.176616097	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 21)
23	5.681972565	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 24)
24	5.695481674	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 23)
25	6.199516585	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 26)
26	6.215834674	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 25)
27	6.721627359	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 28)
28	6.736131766	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 27)
29	7.237180773	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 30)
30	7.249822284	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 29)
31	7.752189573	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 32)
32	7.762660202	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 31)
33	8.263998784	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 34)
34	8.275803813	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 33)
35	8.780832272	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 36)
36	8.795738611	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 35)
37	9.298986980	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 38)
38	9.313349859	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0001, seq=1/256, ttl=115 (request in 37)
39	9.816677263	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (reply in 40)

Zoom in – ping request:

Frame 1: 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface any, id 0
Linux cooked capture v1

Internet Protocol Version 4, Src: 192.168.64.2, Dst: 8.8.8.8

0100 = Version: 4

.... 0101 = Header Length: 20 bytes (5)

Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)

Total Length: 28

Identification: 0xcd2a (52522)

Flags: 0x40, Don't fragment

...0 0000 0000 0000 = Fragment Offset: 0

Time to Live: 64

Protocol: ICMP (1)

Header Checksum: 0x5cfc [validation disabled]

[Header checksum status: Unverified]

Source Address: 192.168.64.2

Destination Address: 8.8.8.8

Internet Control Message Protocol

Type: 8 (Echo (ping) request)

Code: 0

Checksum: 0xf7fd [correct]

[Checksum Status: Good]

Identifier (BE): 1 (0x0001)

Identifier (LE): 256 (0x0100)

Sequence Number (BE): 1 (0x0001)

Sequence Number (LE): 256 (0x0100)

[Response frame: 2]

protocol

Packet type – ping request

Checking the checksum- correctness

Packet id

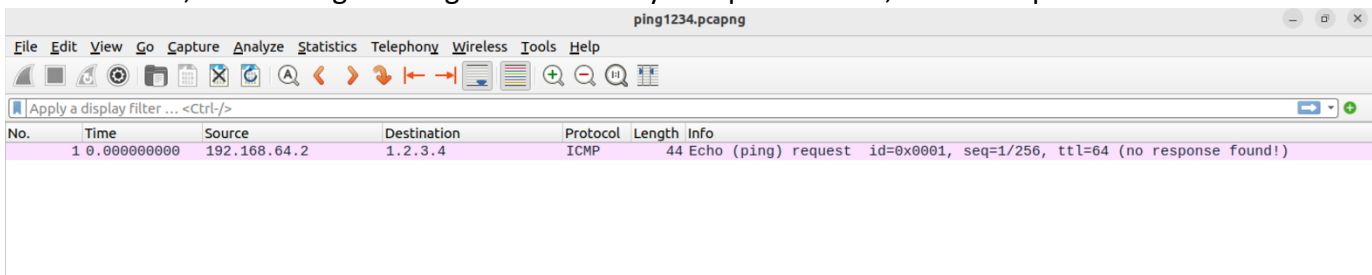
Zoom in- ping reply:

We can observe that the parameters used in the request are the same as those in the ping request.

```
Frame 2: 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface any, id 0
Linux cooked capture v1
Internet Protocol Version 4, Src: 8.8.8.8, Dst: 192.168.64.2
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 28
  Identification: 0x0000 (0)
  Flags: 0x00
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 115
  Protocol: ICMP (1)
  Header Checksum: 0x3727 [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 8.8.8.8
  Destination Address: 192.168.64.2
Internet Control Message Protocol
  Type: 0 (Echo (ping) reply)
  Code: 0
  Checksum: 0xffff [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 1 (0x0001)
  Sequence Number (LE): 256 (0x0100)
  [Request frame: 1]
  [Response time: 12.969 ms]
```

Using an incorrect Ip address (1.2.3.4)

As we can see, when using a wrong IP address only a request is sent, but no response is found



The image shows a Wireshark capture window titled 'ping1234.pcapng'. The packet list shows a single packet: No. 1, Time 0.000000000, Source 192.168.64.2, Destination 1.2.3.4, Protocol ICMP, Length 44, Info 'Echo (ping) request id=0x0001, seq=1/256, ttl=64 (no response found!)'. The packet details pane shows the ICMP Echo (ping) request structure with fields: Type: 8, Code: 0, Checksum: 0xf7fd, Identifier (BE): 1, Identifier (LE): 256, Sequence Number (BE): 1, and Sequence Number (LE): 256.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.64.2	1.2.3.4	ICMP	44	Echo (ping) request id=0x0001, seq=1/256, ttl=64 (no response found!)

Zoom in :

As we can see no response was sent.

```
Frame 1: 44 bytes on wire (352 bits), 44 bytes captured (352 bits) on interface any, id 0
Linux cooked capture v1
Internet Protocol Version 4, Src: 192.168.64.2, Dst: 1.2.3.4
Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0xf7fd [correct]
  [Checksum Status: Good]
  Identifier (BE): 1 (0x0001)
  Identifier (LE): 256 (0x0100)
  Sequence Number (BE): 1 (0x0001)
  Sequence Number (LE): 256 (0x0100)
  [No response seen]
```

Part 2:

Using a correct IP address (8.8.8.8)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 2)
2	0.010391343	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 1)
3	1.008629517	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 4)
4	1.024388158	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 3)
5	2.017413843	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 6)
6	2.028599533	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 5)
7	3.024338028	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 8)
8	3.037239038	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 7)
9	4.033792854	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 10)
10	4.047540921	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 9)
11	5.039300543	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 12)
12	5.053042652	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 11)
13	6.049579620	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 14)
14	6.062025999	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 13)
15	7.057984221	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 16)
16	7.070778147	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 15)
17	8.068836336	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 18)
18	8.084049803	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 17)
19	9.078850534	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 20)
20	9.093689745	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 19)
21	10.089167082	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 22)
22	10.104564095	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 21)
23	11.094575478	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 24)
24	11.110108035	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 23)
25	12.103879370	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 26)
26	12.118347367	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 25)
27	13.111805084	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 28)
28	13.125221063	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 27)
29	14.118226784	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 30)
30	14.131834267	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 29)
31	15.126057522	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 32)
32	15.138694863	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 31)
33	16.132332870	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 34)
34	16.148354894	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 33)
35	17.140969146	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 36)
36	17.153299398	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 35)
37	18.149194092	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 38)
38	18.164011638	8.8.8.8	192.168.64.2	ICMP	44	Echo (ping) reply id=0x0012, seq=1/256, ttl=115 (request in 37)
39	19.152934141	192.168.64.2	8.8.8.8	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (reply in 40)

Using an incorrect Ip address (1.2.3.4)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.64.2	1.2.3.4	ICMP	44	Echo (ping) request id=0x0012, seq=1/256, ttl=64 (no response found!)